

EE414 Term Project Report

Ahmet Emre Eser

Sena Güneş Şirinler

11/01/2025

Table of Contents

EE414 Term Project Report.....	1
Introduction.....	3
Network Topologies.....	3
Section A: Point-to-Point Network Analysis.....	3
Experimental Setup.....	3
Results and Analysis.....	3
Throughput.....	4
Delay (UDP).....	5
Delay (TCP).....	5
Section B: LAN Configuration with TCP Variants.....	6
Experimental Setup.....	6
Analysis of Results.....	6
Throughput Performance.....	8
Delay Characteristics.....	8
Packet Loss.....	8
TCP New Reno and Vegas Comparison and Recommendations.....	9
Appendix.....	10
How to Run.....	10
Code Breakdown.....	11
partA.cc Breakdown.....	11
partB.cc Breakdown.....	12

Introduction

This project investigates the behavior and performance characteristics of two TCP/IP networks under varying load conditions using the NS-3 network simulator. This project analyzes throughput, delay, and packet loss metrics while comparing the performance of TCP NewReno and TCP Vegas congestion control algorithms.

Network Topologies

The project conducts two sets of experiments:

1. Section A: Point-to-Point Network
 - Three-node configuration with controlled traffic generation
 - Systematic load variation from 1 Mbps to 10 Mbps
 - Analysis of throughput and delay metrics
2. Section B: LAN Configuration
 - Extended topology with multiple nodes and a router
 - Implementation of TCP NewReno and TCP Vegas
 - Load variation from 10 Mbps to 100 Mbps
 - Comprehensive analysis of throughput, delay, and packet loss

Section A: Point-to-Point Network Analysis

Experimental Setup

- Network topology: Three-node configuration (Node A → Router B → Node C)
- Link characteristics:
 - Data rate: 5 Mbps
 - Link delay: 2 ms
 - Packet size: 1024 bytes
- Load variation: 1 Mbps to 10 Mbps in 1 Mbps increments

Results and Analysis

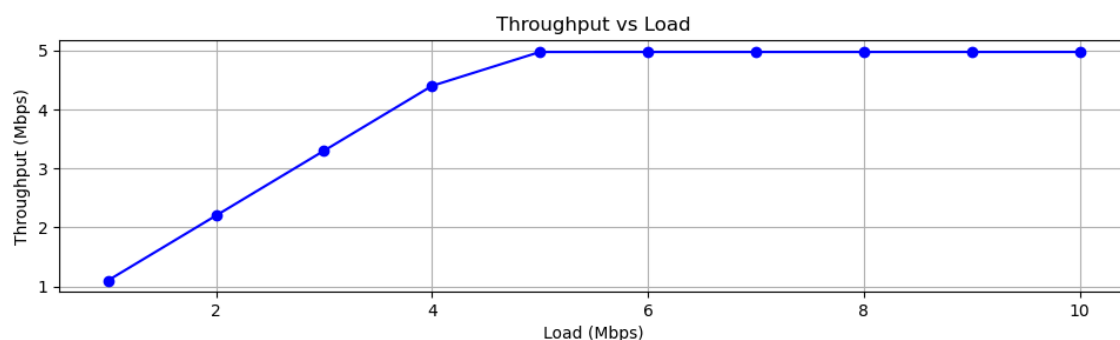


Fig. 1: Throughput vs Load Graph with UDP

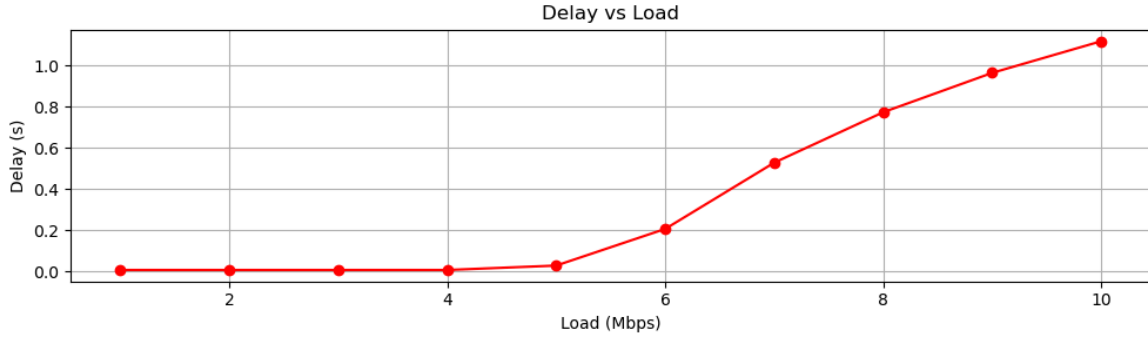


Fig. 2: Delay vs Load Graph for configuration with UDP

Load (Mbps)	Throughput (Mbps)	Delay (ms)
1	1.02751	7.3728
2	2.05334	7.3728
3	3.07917	7.3728
4	4.10499	7.3728
5	4.98483	28.7438
6	4.98483	206.695
7	4.98483	528.309
8	4.98483	773.127
9	4.98483	964.305
10	4.98484	1117.91

Table 1: Metrics obtained for part A. Contents copies from a_metrics.csv file produced by the simulation

Throughput

The throughput vs. load graph shows three distinct regions:

1. Linear Growth (1-4 Mbps): Throughput increases linearly with load
2. Transition (4-5 Mbps): Growth rate decreases as network approaches capacity
3. Saturation (>5 Mbps): Throughput plateaus at approximately 5 Mbps

The network can handle loads below ~4Mbps comfortably (as denoted by the linear relationship between load and TP for 1-4Mbps loads), however channel rate imposes a hard limit on the maximum throughput, as a result network throughput saturates for load values greater than 5Mbps. This behavior supports our theoretical expectations.

Delay (UDP)

The delay vs load graph characteristics show two distinct regions:

1. Stable Region (1-4 Mbps): Low and consistent delay (~0.01s)
2. Approximately linear Increase (>4 Mbps): Delay jumps significantly and increases ~linearly.

The sharp increase in delay coincides with the network reaching its capacity, indicating buffer filling at Router B.

The current configuration of part A uses UDP which does not implement congestion control. When we use TCP, which implements congestion control mechanisms, the delay vs load graph changes significantly. You may find the graphs for runs with TCP below:

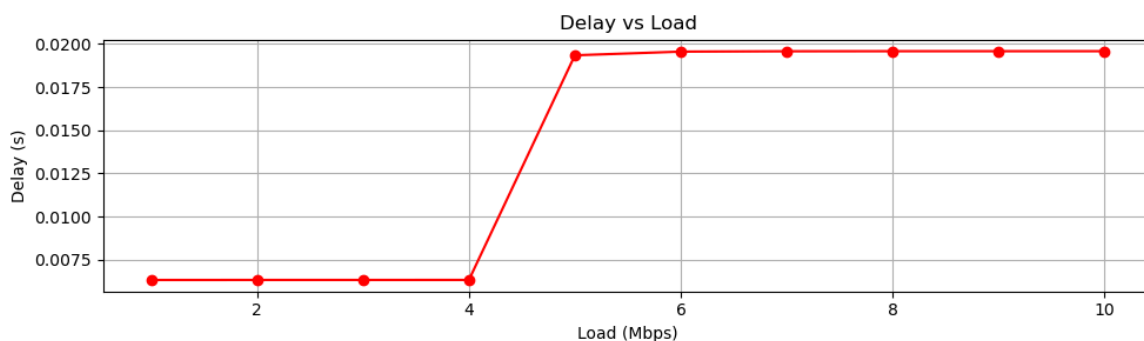


Fig. 3: Delay vs Load Graph (UDP)

Delay (TCP)

The delay vs load graph characteristics show three distinct regions:

1. Stable Region (1-4 Mbps): For low loads, the delay remains constant and very low (~0 ms).
2. Sharp Increase (4-5 Mbps): Delay jumps significantly.
3. Stable Region (>5 Mbps): Delay stabilizes at approximately 20 ms for higher loads

At low loads, the link is underutilized, so packets are transmitted immediately without significant queuing. As the load approaches the link's capacity, packets start to queue at Router B, leading to increased delay. After saturation, the queue reaches its maximum size, and delay stabilizes because packets beyond the buffer are dropped, preventing further delay growth.

The throughput vs load relation remains unchanged however we observe that delay values saturate around 0.02 seconds in TCP. This is likely due to congestion control. UDP does not implement congestion control, hence the delays increase without bound accompanying increases in load past the channel's threshold.

Section B: LAN Configuration with TCP Variants

Experimental Setup

- Extended topology: LAN (A1, A2, A3) → Router B → Node C
- Network parameters:
 - LAN data rate: 100 Mbps
 - LAN delay: 1 ms
 - Packet size: 512 bytes
- TCP variants tested: NewReno and Vegas
- Load variation: 10 Mbps to 100 Mbps in 10 Mbps increments

Analysis of Results

Results obtained in part B are provided below:

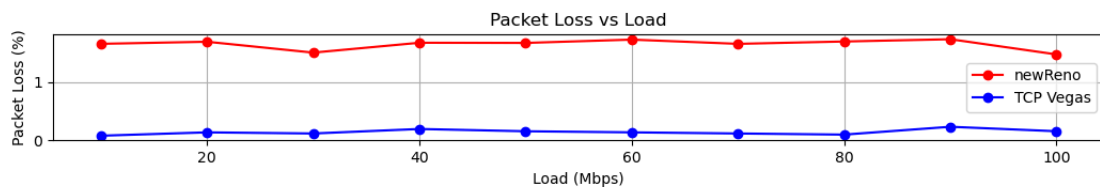


Fig. 4: Packet Loss vs Load Graph

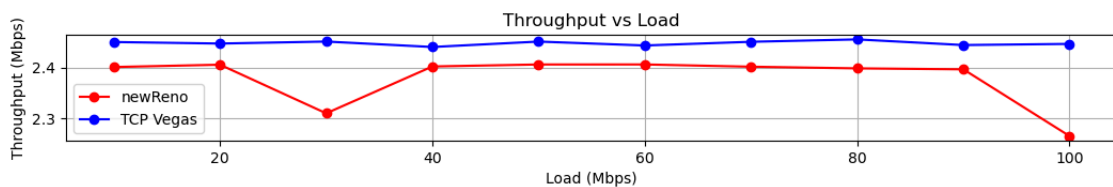


Fig. 5: Throughput vs Load Graph

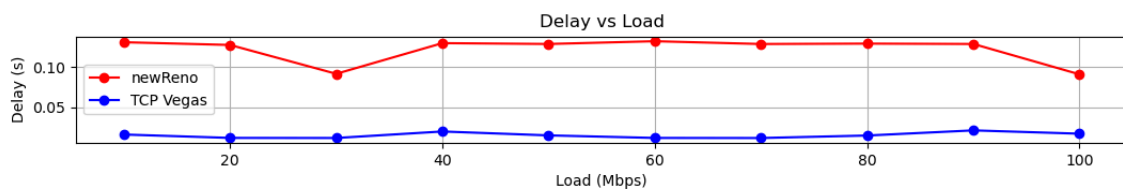


Fig. 6: Delay vs Load Graph

Load (Mbps)	Throughput (Mbps)	Delay (s)	Packet Loss (%)
10	2.40129	0.131236	1.6548
20	2.40589	0.127825	1.69036
30	2.30973	0.0917251	1.50391
40	2.40241	0.130096	1.67308
50	2.40635	0.129053	1.67019
60	2.40641	0.132501	1.72712
70	2.40193	0.129021	1.65448
80	2.39865	0.12951	1.69459
90	2.39676	0.129056	1.73377
100	2.26475	0.0910524	1.47269

Table 2: Metrics for TCP New Reno. Contents copies from b_metrics.csv file produced by the simulation

Load (Mbps)	Throughput (Mbps)	Delay (s)	Packet Loss (%)
10	2.4508	0.0157642	0.0766871
20	2.44776	0.011525	0.134305
30	2.45171	0.0114256	0.114943
40	2.44094	0.0195606	0.192308
50	2.45167	0.014683	0.153198
60	2.44392	0.011493	0.134538
70	2.45124	0.0113961	0.114987
80	2.45596	0.0145081	0.0956389
90	2.44478	0.0209431	0.230282
100	2.44706	0.0167967	0.153521

Table 3: Metrics for TCP Vegas. Contents copies from b_metrics.csv file produced by the simulation

Throughput Performance

1. TCP Vegas:
 - Maintains consistent throughput (~2.45 Mbps)
 - Shows better stability across different loads
 - Less variation in performance
2. TCP NewReno:
 - Slightly lower average throughput (~2.4 Mbps)
 - More variable performance
 - Significant drop at high loads (100 Mbps)

Delay Characteristics

1. TCP Vegas:
 - Lower average delay (~0.02s)
 - More consistent delay across loads
 - Better delay management
2. TCP NewReno:
 - Higher average delay (~0.1s)
 - More variable delay characteristics
 - Shows sensitivity to load changes

Packet Loss

1. TCP Vegas:
 - Minimal packet loss (<0.5%)
 - Consistent performance across loads
 - Better congestion avoidance
2. TCP NewReno:
 - Higher packet loss (~1.5%)
 - More variable loss characteristics
 - Reactive rather than proactive congestion control

TCP New Reno and TCP Vegas differ in how they handle congestion. New Reno is reactive, detecting congestion through packet loss and reducing the congestion window to recover from it. This approach is simple and fair in loss-based networks but inefficient in high-bandwidth scenarios due to its reliance on post-congestion recovery.

TCP Vegas, on the other hand, is proactive, using RTT variations to detect congestion early and adjust the window size to prevent packet loss. This results in smoother throughput and better efficiency in high-bandwidth networks but can lead to fairness issues when competing with more aggressive loss-based protocols. Vegas's complexity stems from its reliance on precise RTT measurements, while New Reno's simplicity makes it more robust in diverse conditions.

TCP New Reno and Vegas Comparison and Recommendations

TCP Vegas demonstrates superior performance in this network configuration:

1. Better stability in throughput
2. Lower and more consistent delay
3. Significantly lower packet loss
4. More efficient congestion management

The proactive approach of TCP Vegas, which uses RTT variations to detect potential congestion, proves more effective than NewReno's reactive loss-based approach. Vegas's ability to maintain stable performance while keeping buffer occupancy low makes it particularly suitable for:

- Networks requiring consistent performance
- Applications sensitive to delay
- Scenarios where packet loss is costly

Appendix

How to Run

You must place each file in a specific folder inside your ns3 simulator in order to run it properly:

- **partA.cc and partB.cc:** placed inside the scratch folder
- **analyze.py and makefile:** placed inside the same folder as the ns3 executable

The proper directory hierarchy should look like this:

Ns3-folder:

- ns3 (executable)
- analyze.py
- makefile
- scratch (folder)
 - ◆ partA.cc
 - ◆ partB.cc

analyze.py generates graphs from csv files produced by the simulation. It takes the following options:

- “**python3 analyze.py -o a**” will generate graphs for only part A
- “**python3 analyze.py -o b**” will generate graphs for only part B
- “**python3 analyze.py -o ab**” will generate graphs for both parts

To run the simulation and generate graphs:

- Run either:
 - “**make all**”
 - or
 - “**./ns3 run partA**” then “**./ns3 run partB**” then “**python3 analyze -o ab**”

You must have GNU make installed to use the “make” command and matplotlib python library to generate graphs with analyze.py. Note that all commands above must be run in the same folder as the ns3 executable (ns3-3.43/) and that graph/ and pcaps/ folder will only be generated if you use the first option and not the second.

partA.cc will use UDP applications by default, if you want to use TCP applications uncomment **#define USE_TCP** at the beginning of partA.cc.

When the simulation is run via “make all” the following files will be generated at the specified locations:

- a_metrics.csv, b_metrics.csv in the same folder as the ns3 executable
- A folder named graphs/ will be created and the graph images will be placed there
- A folder named pcaps/ will be created and all generated .pcap files will be placed there.

The directory hierarchy will look like this after running “make all”

```
ns-3.43/
  → ns3 (executable)
  → analyze.py
  → Makefile
  → a_metrics.csv
  → b_metrics.csv
  → pcap/
  → graphs/
    ◆ graph_b.png
    ◆ graph_b.png
  → scratch/
    ◆ partA.cc
    ◆ partB.cc
```

Running the simulation will replace any existing b_metrics.csv, a_metrics.csv or generated graph files inside the graph folder.

Code Breakdown

Provided below are important snippets from the simulation scripts and explanations highlighting their functionality within the programs.

partA.cc Breakdown

```
OnOffHelper source("ns3::UdpSocketFactory",
InetSocketAddress(interfaces2.GetAddress(1), port)); source.SetAttribute("DataRate",
DataRateValue(DataRate(std::to_string(load) + "Mbps"))); // Set exact load
source.SetAttribute("PacketSize", UintegerValue(1024)); // Packet size in bytes
source.SetAttribute("OnTime", StringValue("ns3::ConstantRandomVariable[Constant=1]"));
// always working
source.SetAttribute("OffTime", StringValue("ns3::ConstantRandomVariable[Constant=0]"));
```

This section configures the UDP traffic generator. It creates a UDP socket, sets the data rate according to the desired load, uses 1024-byte packets, and configures the source to send continuously (OnTime=1, OffTime=0). To configure the script to use TCP instead of UDP, you must replace all instances of “ns3::UdpSocketFactory” with “ns3::TcpSocketFactory”.

```
// Create point-to-point links
PointToPointHelper p2p; p2p.SetDeviceAttribute("DataRate", StringValue("5Mbps")); // Set
high enough to handle 10 Mbps load
p2p.SetChannelAttribute("Delay", StringValue("2ms")); p2p.SetQueue("ns3::DropTailQueue",
"MaxSize", StringValue("10p")); // fifo queue in every connection // Install devices
NetDeviceContainer devices1 = p2p.Install(nodes.Get(0), nodes.Get(1)); // A to B
NetDeviceContainer devices2 = p2p.Install(nodes.Get(1), nodes.Get(2)); // B to C
```

This creates the network topology with 5Mbps point-to-point links, 2ms delay, and 10-packet FIFO queues. It then installs these connections between Node A and Router B, and between Router B and Node C.

partB.cc Breakdown

```
// Create LAN
CsmaHelper csma; csma.SetChannelAttribute("DataRate", StringValue("100Mbps")); // lan
rate 100Mbps
csma.SetChannelAttribute("Delay", StringValue("1ms")); NetDeviceContainer lanDevices =
csma.Install(lanNodes); // Create point-to-point links
PointToPointHelper p2p; p2p.SetDeviceAttribute("DataRate", StringValue("100Mbps")); //
router & dest conn. rate: 100Mbps
p2p.SetChannelAttribute("Delay", StringValue("1ms"));
```

This creates the network topology for part B. The LAN segment uses CSMA with 100Mbps bandwidth and 1ms delay. The point-to-point connections from the LAN to router and router to destination also use 100Mbps links with 1ms delay.

```
// Install Internet stack with TCP variant
InternetStackHelper internet; Config::SetDefault("ns3::TcpL4Protocol::SocketType",
StringValue("ns3::" + tcpVariant));
```

This configures the TCP variant (either NewReno or Vegas) for the simulation. The TCP variant is set globally before installing the network stack.

```
double simulationTime = (iter->second.timeLastRxPacket.GetSeconds() -
iter->second.timeFirstTxPacket.GetSeconds()); double throughput = ((iter->second.rxBytes *
8.0) / simulationTime) * 0.000001f; // Throughput in Mbps
double delay = (iter->second.delaySum.GetSeconds() / iter->second.rxPackets); // Average
delay in seconds
double packetLoss = ((double)(iter->second.txPackets - iter->second.rxPackets) /
iter->second.txPackets) * 100.0f;
```

This calculates the key performance metrics:

- Throughput: Total received bits divided by simulation time
- Average Delay: Total delay divided by number of received packets
- Packet Loss: Percentage of packets that didn't make it to the destination